

P R I S M A × E - C O M M E R C E

Prisma สำหรับจัดการข้อมูล ในระบบ E-commerce

ตั้งแต่ *Database Design* ถึง *CRUD Operations*

Prisma Schema

Prisma Client

Migrations

SQL Server

Workshop สำหรับทีม Backend • Node.js + JavaScript

24 เม.ย. 2026

Prisma คืออะไร?

Next-generation ORM สำหรับ Node.js

- ORM ยุคใหม่สำหรับ Node.js — ใช้ได้ทั้ง JavaScript และ TypeScript
- จัดการฐานข้อมูลด้วย schema เดียว อ่านง่าย
- ลด SQL boilerplate — เขียนน้อย ได้มาก
- Auto-generate Prisma Client พร้อม autocomplete ใน VSCode
- รองรับ SQL Server, PostgreSQL, MySQL, SQLite, MongoDB

Prisma Schema

ไฟล์เดียวกำหนด model, relation, datasource

Prisma Client

Query API เรียกใช้งานใน Node.js ได้ทันที

Prisma Migrate

จัดการ versioning ของฐานข้อมูลอัตโนมัติ

ภาพรวมระบบ E-commerce

โมดูลหลักและ flow การสั่งซื้อของลูกค้า



ลูกค้าเลือกสินค้า → ใส่ตะกร้า → สั่งซื้อ → ชำระเงิน → ระบบอัปเดต stock และสร้างใบเสร็จ

Modules หลักของระบบ

User

Product

Category

Cart / CartItem

Order / OrderItem

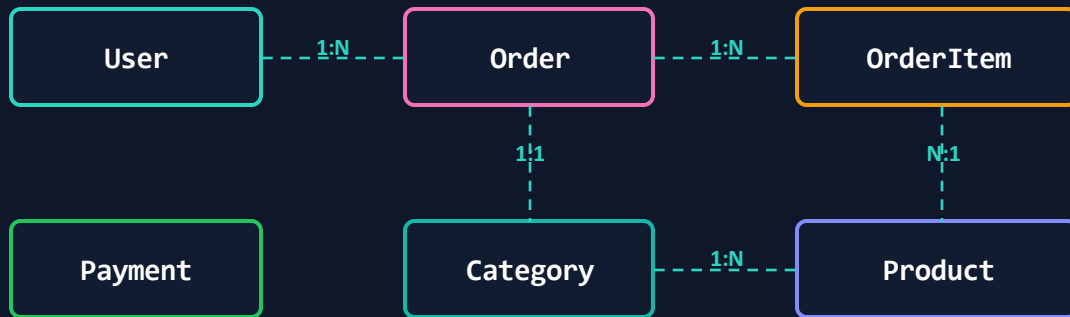
Payment

การออกแบบฐานข้อมูล

Entity หลักและความสัมพันธ์แบบ ER

Entity หลัก

User	Category
Product	Cart
CartItem	Order
OrderItem	Payment



User 1:N Order • Category 1:N Product • Order 1:N OrderItem • Product 1:N OrderItem • Order 1:1 Payment

การติดตั้ง Prisma

เริ่มต้นโปรเจกต์ Node.js + Prisma ภายใน 1 นาที

terminal

```
# 1) สร้างโปรเจกต์
npm init -y

# 2) ติดตั้ง Prisma
npm install prisma @prisma/client
npx prisma init --datasource-provider sqlserver

#หมายเหตุ: ใช้ JavaScript ธรรมดา
```

.env

```
// .env
DATABASE_URL="sqlserver://localhost:1433;
database=ecom;user=sa;password=YourPass123;
trustServerCertificate=true"
```

ไฟล์ที่ถูกสร้าง

prisma/schema.prisma

นิยาม model และ datasource

.env

เก็บ DATABASE_URL แบบลับ

node_modules/@prisma/client

Client ที่ถูก generate

package.json

dependencies ของโปรเจกต์

ตัวอย่าง Prisma Schema

ไฟล์เดี่ยวครอบคลุม *datasource + model + relation*

prisma/schema.prisma

```
datasource db {
  provider = "sqlserver"
  url      = env("DATABASE_URL")
}

model Category {
  id        Int        @id @default(autoincrement())
  name      String     @unique
  products Product[]
}

model Product {
  id          Int          @id @default(autoincrement())
  name        String
  price       Decimal     @db.Decimal(10, 2)
  stock       Int         @default(0)
  categoryId  Int
  category    Category    @relation(fields: [categoryId], references: [id])
  items       OrderItem[]
  createdAt   DateTime    @default(now())
}
```

Prisma ORM • Node.js (JavaScript) • SQL Server

Attribute ที่ใช้บ่อย

@id

กำหนด primary key

@default(...)

ค่า default เช่น now(), autoincrement()

@unique

บังคับค่า unique เช่น email

@relation

เชื่อม model ผ่าน foreign key

@db.Decimal

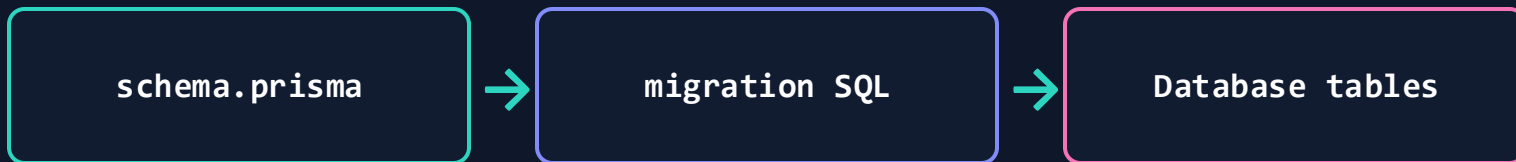
ระบุชนิดฝั่ง database

DateTime

ชนิดวันเวลา

สร้าง Database และ Migration

จาก `schema.prisma` สู่ตารางจริงในฐานข้อมูล



terminal

```
# สร้าง migration ชุดแรก
npx prisma migrate dev --name init

# deploy ไป production
npx prisma migrate deploy
```

Tip: คุณสามารถใช้ migrations ใน dev ได้ แต่ห้ามลบใน production!

```
npx prisma migrate status
```

- แต่ละ migration เป็น .sql ใน `prisma/migrations/`
- เก็บลง git เพื่อให้ทีมซิงก์ schema ตรงกัน
- Production ใช้ `migrate deploy` เท่านั้น
- แก้ไขย้อนหลังด้วย `migrate resolve` ถ้าติดปัญหา

การ Seed ข้อมูลเริ่มต้น

เตรียมข้อมูลจำลองสำหรับ *dev* และ *test*

ทำไมต้อง Seed?

- มีข้อมูลตัวอย่างให้ dev ทดสอบทันที
- ใช้เป็น baseline ใน E2E test / CI
- รีเซ็ต database ใหม่ก็ได้ข้อมูลเดิม
- ช่วย onboarding คนเข้าทีมใหม่ได้เร็ว

```
npx prisma db seed
```

prisma/seed.js

```
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

async function main() {
  const tech = await prisma.category.create({
    data: { name: "Technology" },
  });

  await prisma.product.createMany({
    data: [
      { name: "Keyboard", price: 1290, stock: 20, categoryId:
tech.id },
      { name: "Mouse", price: 690, stock: 35, categoryId:
tech.id },
    ],
  });

  await prisma.user.create({
    data: { email: "demo@shop.io", name: "Demo User" },
  });
}
```

Create Data

เพิ่มข้อมูลด้วย *Prisma Client*

●●● create.js

```
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();
```

// 1) สร้างสินค้าใหม่

```
const product = await prisma.product.create({
  data: {
    name: "Mechanical Keyboard",
    price: 2490,
    stock: 10,
    categoryId: 1,
  },
});
```

// 2) สร้าง order พร้อม order items (nested)

```
const order = await prisma.order.create({
  data: {
    userId: 1,
    total: 2980,
    items: {
```

Key API

create()

สร้าง 1 record — return record พร้อม id

createMany()

insert หลายแถวพร้อมกัน เร็วและประหยัด

nested create

สร้าง parent + children ในคำสั่งเดียว

include: true

return ข้อมูลที่เชื่อมกลับมาด้วย

skipDuplicates

ข้ามแถวที่ unique ซ้ำ (ไม่ error)

Read Data

Query ข้อมูลด้วย *findMany*, *include*, *where*

```
●●● read.js

// 1) ดึงสินค้าทั้งหมด
const all = await prisma.product.findMany();

// 2) ดึงสินค้าตามหมวดหมู่ + เรียงราคาต่ำ→สูง
const tech = await prisma.product.findMany({
  where: { category: { name: "Technology" }, stock: { gt: 0 } },
  orderBy: { price: "asc" },
  select: { id: true, name: true, price: true },
  take: 10,
});

// 3) ดึง order พร้อมรายการสินค้า + ชื่อสินค้า
const order = await prisma.order.findUnique({
  where: { id: 42 },
  include: {
    user: { select: { name: true, email: true } },
    items: { include: { product: true } },
  },
});
```

Prisma ORM • Node.js (JavaScript) • SQL Server

Query Operators

findMany	หลาย record
findUnique	1 record ตาม key
include	join ข้อมูล relation
select	เลือกเฉพาะ field
where	ตัวกรอง (gt, lt, contains)
orderBy	จัดเรียง asc / desc
take / skip	pagination

Update Data

แก้ไขข้อมูลแบบแถวเดียวหรือทีละหลายแถว

● ● ● update.js

```
// 1) อัปเดตราคาและลด stock ของสินค้า
await prisma.product.update({
  where: { id: 1 },
  data: {
    price: 1390,
    stock: { decrement: 1 }, // atomic
  },
});
```

```
// 2) เปลี่ยนสถานะ order
await prisma.order.update({
  where: { id: 42 },
  data: { status: "PAID" },
});
```

```
// 3) อัปเดตหลายแถว – ลดราคาทุกสินค้า 10%
await prisma.product.updateMany({
  where: { stock: { gt: 100 } },
  data: { price: { multiply: 0.9 } },
});
```

update() vs updateMany()

update() ต้องระบุ unique key; updateMany() ขึ้นจำนวนที่แก้

Atomic Operations

ใช้ increment, decrement, multiply เพื่อกัน race condition

upsert() Pattern

เหมาะกับ sync master data หรือ webhook ที่ส่งซ้ำได้

Delete Data

Hard delete, soft delete และข้อควรระวังเรื่อง relation

delete.js

```
// 1) Hard delete - ลบถาวร
await prisma.orderItem.delete({
  where: { id: 10 },
});

// 2) ลบหลายแถว
await prisma.product.deleteMany({
  where: { stock: 0, createdAt: { lt: lastYear } },
});

// 3) Soft delete - เพิ่ม field deletedAt
await prisma.product.update({
  where: { id: 1 },
  data: { deletedAt: new Date() },
});

// ต้องกรอง query ทุกครั้ง
await prisma.product.findMany({
  where: { deletedAt: null },
});
```

Prisma Client Node.js (v4.15.0) - SQL Server

Hard Delete

- ลบจริงออกจากรายการ
- กู้คืนไม่ได้
- ระวัง FK constraint
- ใช้กับข้อมูลชั่วคราว

Soft Delete

- ตั้ง flag deletedAt
- กู้คืนได้ง่าย
- เก็บ history / audit
- ใช้กับข้อมูลสำคัญ

⚠ ถ้ามี relation → ต้องตั้ง onDelete: Cascade / SetNull ใน SetNull ใน schema

Transaction & Business Logic

ทั้งหมดสำเร็จพร้อมกัน หรือย้อนกลับพร้อมกัน (ACID)

● ● ● checkout.js

```
// checkout: สร้าง order, ลด stock, บันทึก payment
const result = await prisma.$transaction(async (tx) => {
  // 1) สร้าง order พร้อม items
  const order = await tx.order.create({
    data: {
      userId, total,
      items: { create: items },
    },
  });

  // 2) ลด stock ของแต่ละ product (atomic)
  for (const it of items) {
    await tx.product.update({
      where: { id: it.productId },
      data: { stock: { decrement: it.quantity } },
    });
  }

  // 3) บันทึก payment
  await tx.payment.create({
    data: {
      orderId, total,
    },
  });
});
```

ทำไมต้องใช้ Transaction เป็นก้อน

เดียว

- CONSISTENT — ข้อมูลสม่ำเสมอ
- ISOLATED — ไม่ชนกับ transaction อื่น

2 รูปแบบ

Interactive: `$transaction(async tx => { ... })`

Sequential: `$transaction([q1, q2, q3])`

Error Handling

จับ error ของ Prisma ให้ทำงานและพร้อมส่งออก API

● ● ● userService.js

```
const {
  PrismaClientKnownRequestError,
  PrismaClientValidationError,
} = require("@prisma/client/runtime/library");

async function createUser(data) {
  try {
    return await prisma.user.create({ data });
  } catch (err) {
    if (err instanceof PrismaClientKnownRequestError) {
      if (err.code === "P2002") {
        // unique constraint (เช่น email ซ้ำ)
        throw { status: 409, message: "Email นี้ถูกใช้งานแล้ว" };
      }
    }
    if (err.code === "P2025") {
      // record not found
      throw { status: 404, message: "ไม่พบข้อมูล" };
    }
  }
}
```

Prisma ORM • Node.js (JavaScript) • SQL Server
if (err instanceof PrismaClientValidationError) {
 throw { status: 400, message: "ข้อมูลไม่ถูกต้อง" };

Error Code ที่เจอบ่อย

P2002

Unique constraint failed

P2003

Foreign key constraint failed

P2025

Record ไม่พบสำหรับ update/delete

P2014

Required relation violation

P1001

Cannot reach database server

✓ ควรมี central errorHandler middleware แปลง error → error → HTTP response มาตรฐาน

Enum & Indexing

ทำให้ *schema* อ่านง่าย และ *query* เร็วขึ้น

Enum — ค่าที่เป็นไปได้แบบจำกัด

```
schema + order.js

enum OrderStatus {
  PENDING
  PAID
  SHIPPED
  DELIVERED
  CANCELLED
}

model Order {
  id      Int      @id @default(autoincrement())
  userId  Int
  status  OrderStatus @default(PENDING)
  total   Decimal   @db.Decimal(12, 2)
}

// ใช้กันใน Node.js
await prisma.order.update({
  where: { id: 1 },
  data: { status: "SHIPPED" }
})
```

Prisma ORM • Node.js (JavaScript) • SQL Server

Index — เร่งความเร็วในการ query

```
schema.prisma

model Product {
  id          Int      @id @default(autoincrement())
  name        String
  categoryId  Int
  price       Decimal   @db.Decimal(10, 2)
  createdAt   DateTime  @default(now())

  @@index([categoryId])
  @@index([name, createdAt])
  @@unique([categoryId, name])
}
```

เมื่อไหร่ควรสร้าง Index?

- field ที่ใช้ใน `where/join/orderBy` บ่อย
- foreign key
- composite ตามลำดับใช้งาน

Middleware (\$use)

แทรก logic ก่อน/หลังทุก query — ใช้ทำ soft-delete auto-filter

●●● softDelete.middleware.js

```
const { PrismaClient } = require("@prisma/client");
const prisma = new PrismaClient();

// แทรก middleware ก่อนส่ง query ไปยัง database
prisma.$use(async (params, next) => {
  // 1) แปลง delete → update(deletedAt)
  if (params.model === "Product") {
    if (params.action === "delete") {
      params.action = "update";
      params.args.data = { deletedAt: new Date() };
    }
    if (params.action === "deleteMany") {
      params.action = "updateMany";
      params.args.data = { ...(params.args.data || {}), deletedAt: new Date() };
    }
  }

  // 2) ซ่อน record ที่ถูกลบจาก findMany / findFirst
  if (["findMany", "findFirst"].includes(params.action)) {
    params.args.where = {
      ...((params.args.where || {})),
    };
  }

  return next(params);
});
```

Request Pipeline

Your Code

prisma.product.delete()



Middleware

แปลงเป็น update



Prisma Engine

ส่ง SQL ไป DB

⚠ ตั้ง middleware ก่อน query แรกเสมอ

Performance & N+1 Query Problem

ปัญหาคลาสสิกของ ORM และวิธีแก้ด้วย Prisma

✗ Bad — N+1 queries

● ● ● slow.js

```
// 1 query ดึง orders
const orders = await prisma.order.findMany();

// + N query ดึง user ของแต่ละ order
for (const o of orders) {
  const u = await prisma.user.findUnique({
    where: { id: o.userId },
  });
  o.user = u;
}
```

// 100 orders = 101 queries ⚠

✓ Good — single query + include

● ● ● fast.js

```
// 1 query – JOIN ให้เรียบร้อย
const orders = await prisma.order.findMany({
  include: {
    user: { select: { id: true, name: true } },
    items: { include: { product: true } },
  },
});
// 100 orders = 1 query ✓
```

เทคนิคเพิ่ม Performance

include / select

join เท่าที่จำเป็น

Pagination

take / skip / cursor

\$transaction([])

batch หลาย query

Add Index

เรียง where / orderBy

Log Slow Query

log: ['query'] debug ได้

Raw SQL (\$queryRaw / \$executeRaw)

เมื่อ Prisma API ไม่พอ — ใช้ SQL ตรงแบบปลอดภัย

rawSql.js

```
// 1) $queryRaw - SELECT, return rows
// ใช้ tagged template เพื่อ parameterize อินพุต (กัน SQL injection)
const email = "demo@shop.io";
const users = await prisma.$queryRaw`
  SELECT TOP 10 id, name, email
  FROM [User]
  WHERE email = ${email}
`;
```

```
// 2) $executeRaw - INSERT/UPDATE/DELETE, return affected rows
const minStock = 0;
const n = await prisma.$executeRaw`
  UPDATE [Product]
  SET stock = stock + 10
  WHERE stock <= ${minStock}
`;
```

```
// 3) Dynamic SQL ด้วย Prisma.sql (compose ได้)
const { Prisma } = require("@prisma/client");
const where = Prisma.sql`WHERE status = ${"PAID"}`;
```

```
const rows = await prisma.$queryRaw`
  SELECT COUNT(*) FROM [Order] ${where}
`;
```

ใช้เมื่อไหร่?

Aggregation ซับซ้อน

window function, CTE, pivot

Vendor-specific

SQL Server: MERGE, FOR XML

Performance tuning

hint, index override

Legacy schema

ตารางที่ไม่ได้จัดการโดย Prisma

Bulk import

BULK INSERT / stored proc

**⚠ ห้ามใช้ \$queryRawUnsafe กับ input จากผู้ใช้ — เสี่ยง
เสี่ยง SQL injection**

Best Practices

แนวทางที่ควรทำและข้อควรระวัง

1

Service Layer

แยก logic ออกจาก route/controller เพื่อ
test ง่าย

2

Validate Input

ใช้ Joi / Zod / Valibot ก่อนส่งเข้า Prisma

3

Select เฉพาะที่จำเป็น

ลด payload และเพิ่มความเร็ว

4

Handle Errors

จับ Prisma error codes (P2002, P2025)
P2025)

5

Migration Discipline

commit ทุกครั้ง ไม่แก้ไฟล์ย้อนหลัง

6

Transaction

ใช้กับงาน multi-step ที่ต้อง all-or-nothing
nothing

สรุปและก้าวต่อไป

Recap + Next Step สำหรับโปรเจกต์จริงของคุณ

Recap

- Prisma ทำให้การจัดการฐานข้อมูลเร็วและปลอดภัย
- Schema เดี่ยวครอบคลุม model, relation, migration
- CRUD ครบชุด + nested + atomic operations
- Error handling + Enum + Index = ใกล้ production
- Transaction = หัวใจของ e-commerce checkout

NEXT STEP

Authentication

เพิ่ม JWT / OAuth ให้ User

Pagination & Filtering

cursor-based + dynamic where

REST / GraphQL API

Express, Fastify, หรือ Apollo

Testing & CI

Jest + Prisma Testcontainers